Lincoln University College

Faculty of Computer Science and Multimedia

CampusEase: A Web-Based College Automation System for Academic and Administrative Management

FINAL YEAR PROJECT REPORT

Submitted to

Department of Information Technology
Phoenix College of Management
Maitidevi, Kathmandu

In partial fulfillment of the requirements for the degree of
Bachelor in Information Technology (BIT)

Submitted by

Bhupendra Malla
BIT 8th Semester
University ID: LC0003001648

Under the Supervision of

Mr. Saishab Bhattarai
Project Supervisor

May, 2026

Lincoln University College
Faculty of Computer Science and Multimedia
Phoenix College of Management, Maitidevi, Kathmandu

LETTER OF APPROVAL

This is to certify that this project report titled "CampusEase: A Web-Based College Automation System for Academic and Administrative Management" prepared and submitted by Bhupendra Malla (University ID: LC0003001648) in partial fulfillment of the requirements for the degree of Bachelor in Information Technology (BIT) has been reviewed and approved.

The project has been carried out under supervision and meets the academic standards required by the Department of Information Technology, Phoenix College of Management.

Mr. Saishab Bhattarai Project Supervisor Department of Information Technology	**Project Coordinator** Department of Information Technology Phoenix College of Management
Internal Examiner	**External Examiner**

SUPERVISOR'S CERTIFICATE

This is to certify that the student Bhupendra Malla has completed the Bachelor in Information Technology Final Year Project titled "CampusEase: A Web-Based College Automation System for Academic and Administrative Management" under the supervision of Mr. Saishab Bhattarai at Lincoln University College, Faculty of Computer Science and Multimedia.

The student has independently designed and implemented a working software prototype, conducted testing, and documented the system according to academic standards. To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any degree.

I recommend this project report for evaluation.

Signature: _____

Mr. Saishab Bhattarai
Project Supervisor
Department of Information Technology
Phoenix College of Management

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed directly and indirectly to the successful completion of this project. Their guidance, encouragement, and continuous support have been invaluable throughout the entire course of this work.

First and foremost, I would like to express my profound gratitude to the Principal of Phoenix College of Management, Prof. Dr. Fatta Bahadur KC, for his inspiring leadership, academic guidance, and continuous encouragement in fostering an environment dedicated to learning, research, and innovation.

I would also like to extend my heartfelt appreciation to my project supervisor, Mr. Saishab Bhattarai, for his invaluable guidance, constructive feedback, encouragement, and continuous support throughout the development of this project. His expertise, patience, and insightful suggestions greatly contributed to the successful completion of this work.

Furthermore, I am deeply thankful to the Department of Information Technology at Phoenix College of Management for providing the necessary academic environment, institutional support, resources, and facilities required for the successful execution of this project.

I would also like to acknowledge the valuable support and encouragement provided by the faculty members, whose suggestions and assistance played a significant role during the course of this work.

Finally, I express my deepest gratitude to my family and friends for their unwavering support, motivation, patience, and encouragement throughout this journey. Their constant belief in me provided the strength and determination necessary to accomplish this project successfully.

Bhupendra Malla
BIT 8th Semester
University ID: LC0003001648

ABSTRACT

Colleges run on a large number of overlapping manual processes: enrollment, attendance, assignments, examinations, fee collection, scheduling, and day-to-day communication between students, faculty, and administrative staff. Handling these on paper and across disconnected tools is slow, error-prone, and difficult to track. CampusEase is a web-based college automation system that brings these activities together on a single platform. It provides role-based access for students, faculty, secretaries, and administrators; supports subject enrollment and class scheduling; records attendance manually, through a one-time password, or through face recognition; manages assignment distribution and submission, internal marks entry, and automatic internal-marks calculation; and integrates online fee payment through the Khalti gateway with an administrative approval workflow. Real-time messaging, discussion forums, events, clubs, a job-and-CV portal, sponsorship requests, and digital ID-card generation are also included. The system is built with an Angular single-page frontend, a Node.js and Express REST API with Socket.io for real-time features, and a MongoDB database accessed through the Mongoose object-document mapper, with JWT authentication and bcrypt password hashing. The result is a reliable, easy-to-use platform that reduces administrative workload, minimizes errors, and improves transparency and communication across the academic community.

Keywords: College Automation, Role-Based Access Control, Face Recognition Attendance, Online Fee Payment, Real-Time Communication.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BIT	Bachelor in Information Technology
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity Relationship
ERP	Enterprise Resource Planning
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
NoSQL	Not Only Structured Query Language
ODM	Object Document Mapper
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SIS	Student Information System
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
UI	User Interface
UML	Unified Modeling Language
VCS	Version Control System

Chapter 1:

INTRODUCTION

1.1 Introduction

Educational institutions handle an enormous amount of routine work every day. Students enroll in subjects, attend classes, submit assignments, sit examinations, and pay fees. Faculty members prepare schedules, take attendance, distribute and grade assignments, and record marks. Administrative staff manage accounts, approve payments, maintain departments, organize events, and issue documents such as identity cards. Traditionally, most of this work is done manually, on paper, or across a collection of disconnected applications and spreadsheets.

This manual, fragmented approach is slow and error-prone. Records are duplicated or lost, attendance and marks are tedious to compile, fee tracking is hard to reconcile, and communication between students, faculty, and staff relies on notice boards and informal channels. As colleges grow and as students increasingly expect digital, mobile-friendly services, these problems become more pronounced.

A college automation system addresses this by bringing the core academic and administrative activities of an institution onto a single, integrated platform. Instead of separate, manual processes, a well-designed system gives each kind of user a tailored interface, keeps a single authoritative copy of every record, automates repetitive calculations, and enables instant communication. The benefits are reduced workload, fewer errors, faster access to information, and greater transparency for everyone involved.

CampusEase is such a system. It is a web-based platform that unifies enrollment, scheduling, attendance, assignments, examinations and marks, fee payment, and communication, with role-based access for students, faculty, secretaries, and administrators. It adds modern conveniences such as face-recognition attendance, online fee payment through a local payment gateway, real-time messaging, and digital ID-card generation, all behind a clean interface that requires no special technical training to use.

1.2 Problem Statement

Most colleges still rely on manual or loosely connected processes to run their academic and administrative operations. This creates several concrete problems that CampusEase is designed to solve:

- **Fragmented information.** Student records, attendance, marks, fees, and communication are spread across paper registers, spreadsheets, and unrelated tools, with no single source of truth.
- **Manual, repetitive work.** Taking attendance, compiling internal marks, reconciling fee payments, and preparing reports by hand is slow and consumes staff and faculty time that could be better spent.
- **Error and inconsistency.** Re-entering the same data in multiple places leads to mistakes, and there is often no reliable audit trail of who changed what.
- **Weak communication.** Important information reaches students through notice boards or third-party messaging, so updates are easy to miss and hard to track.
- **Limited access control.** Without a structured permission model, it is difficult to ensure that each user, from a student to a finance officer, sees and does only what their role allows.

There is a clear need for an integrated, role-aware web platform that centralizes academic and administrative data, automates routine tasks such as attendance and marks calculation, supports secure online fee payment, and provides direct, real-time communication, all through a simple interface.

1.3 Objectives

1.3.1 General Objective

To design and develop a web-based college automation system that centralizes academic and administrative operations and provides role-based, automated services to students, faculty, and administrative staff.

1.3.2 Specific Objectives

- To design a role-based access control model covering students, faculty, secretaries, and administrators.
- To provide multiple attendance methods, including manual, one-time-password, and face-recognition attendance.
- To integrate secure online fee payment through the Khalti gateway with an administrative approval workflow.

1.4 Significance of the Study

CampusEase demonstrates how a single, well-structured web application can replace a collection of manual and disconnected processes in a college. For students, it offers a single portal for enrollment, attendance, assignments, marks, fees, and communication. For faculty, it automates scheduling, attendance, and marks-related tasks. For administrative staff, it provides structured user management, fee approval, and reporting. Beyond the institution it serves, the project is a practical case study in building a real, multi-role web system with modern technologies, secure authentication, real-time features, and third-party payment integration, and is therefore valuable both as a deployable product and as an academic learning outcome.

1.5 Scope and Limitation

1.5.1 Scope

CampusEase is an integrated college automation platform. Its scope includes:

- Role-based registration and authentication for students, faculty, secretaries, and administrators, with email verification and one-time-password support.
- Subject enrollment using enrollment keys, class scheduling, and academic-record management.
- Attendance recording through manual entry, one-time password, and face recognition.
- Assignment distribution and submission, marks entry, and automatic internal-marks calculation from assignment and attendance scores.
- Online fee payment through the Khalti gateway, with payment status tracking and administrative approval.
- Real-time one-to-one messaging, discussion forums, events, and club membership.
- A job-and-CV portal, sponsorship requests, digital ID-card generation, feedback collection, and dashboards.

1.5.2 Limitation

- The system is a web application intended for use within a single institution; it is not a multi-tenant product serving many colleges from one deployment.

- Face-recognition attendance depends on adequate lighting and a working camera, and is intended as a convenience method alongside manual and one-time-password attendance rather than a biometric-grade security control.
- Online payment is integrated with the Khalti gateway; other payment methods are recorded but processed manually by staff.
- The current deployment uses a single MongoDB instance suitable for an institution-scale workload and academic demonstration rather than very large, high-concurrency, multi-campus environments.
- Authentication uses JWT with bcrypt-hashed passwords appropriate for the project; a large production deployment would add measures such as refresh-token rotation and centralized session management.

1.6 Development Methodology

The project was developed using an Agile methodology based on short, iterative sprints. Because CampusEase is made up of many distinct but related modules (authentication, enrollment, attendance, assignments, marks, fees, communication, and administration), an incremental, feature-by-feature approach was a natural fit. Each sprint delivered one or more working modules: requirements were refined, the data model and API endpoints were built on the backend, the corresponding Angular screens were developed, and the feature was tested before moving on. The role-based access model and the user data model were built early because almost every later feature depends on them. Version control with Git and GitHub was used throughout, allowing changes to be tracked incrementally and modules to be integrated continuously. This iterative style made it possible to absorb feedback and add modules without disrupting features already completed.

1.7 Report Organization

This report is organized into five chapters:

- **Chapter 1: Introduction** presents the background, problem statement, objectives, significance, scope and limitations, development methodology, and the organization of the report.
- **Chapter 2: Background Study and Literature Review** reviews college automation concepts and existing systems and positions CampusEase against them.

- **Chapter 3: System Analysis and Design** presents the feasibility study, requirements, use-case and data models, data flow diagrams, and system design.
- **Chapter 4: Implementation and Testing** describes the implementation of each module and the testing approach and results.
- **Chapter 5: Conclusion and Future Recommendations** concludes the report and discusses lessons learned and future enhancements.

Chapter 2:

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

College Automation and Student Information Systems. A college automation system, sometimes called a campus management or student information system (SIS), is software that digitizes and integrates an institution's academic and administrative processes. Rather than maintaining separate records for admissions, attendance, examinations, and finance, such a system keeps one connected set of data and exposes it through role-specific interfaces. The expected benefits, widely reported in the literature, are reduced manual effort, fewer data-entry errors, faster reporting, and improved communication among stakeholders.

Learning Management and ERP Systems. Two related families of systems are relevant. Learning Management Systems (LMS) such as Moodle and Google Classroom focus on the teaching and learning workflow: distributing course material, collecting assignments, and grading. Enterprise Resource Planning (ERP) systems for education go further, covering finance, human resources, and operations alongside academics, but they are large, costly, and aimed at big institutions. CampusEase sits between these: it covers the academic workflow like an LMS while also handling administrative tasks such as fee payment, user management, and document issuance, but it remains lightweight and tailored to a single college.

Web Application Architecture. Modern web applications are commonly built as a single-page application (SPA) frontend communicating with a backend API over HTTP, often complemented by WebSocket connections for real-time features. CampusEase follows this pattern with an Angular SPA, an Express REST API, and Socket.io for live messaging. Data is stored in a NoSQL document database, MongoDB, accessed through the Mongoose object-document mapper, which suits the varied and evolving document shapes of a college's records.

2.2 Literature Review

Several established systems address parts of the same problem space:

- **Google Classroom** [8] is a free, widely used platform for distributing material, posting assignments, and grading. It is simple and reliable but is focused on the classroom workflow and does not handle administrative concerns such as fee payment, attendance registers, or institutional user management.
- **Moodle** [7] is a mature, open-source LMS that is highly configurable and extensible through plugins. It is powerful for course delivery, but it is heavy to host and administer and is not designed as an end-to-end college administration system.
- **Microsoft Teams for Education** combines communication, meetings, and assignment features. It excels at communication and collaboration but, like the others, is not a complete academic-plus-administrative platform and depends on the wider Microsoft ecosystem.
- **Commercial campus ERPs** provide comprehensive coverage of academics, finance, and operations, but they are expensive, complex to deploy, and far larger than a single college usually needs.

Table 2.1 compares these systems against the needs CampusEase targets.

Table 2.1: Comparison of existing systems with the needs CampusEase targets.

Criterion	**Google Classroom**	**Moodle**	**Campus ERP**	**CampusEase**
Cost	Free	Free (self-host)	Costly	Free
Setup effort	Low	High	High	Low
Academic workflow	Yes	Yes	Yes	Yes
Administration (fees, users)	No	Partial	Yes	Yes
Attendance register	No	Plugin	Yes	Yes (manual/OTP/-face)
Online fee payment	No	No	Yes	Yes (Khalti)
Real-time messaging	Limited	Limited	Varies	Yes (Socket.io)

The common limitation is clear: learning management tools cover teaching but not administration, while full ERPs cover everything but are costly and complex. Neither

is well matched to a single college that wants both academic and administrative automation in one affordable, easy-to-run package. CampusEase addresses this gap by combining an LMS-style academic workflow with administrative features, role-based access for every type of user, modern conveniences such as face-recognition attendance and online payment, and real-time communication, all on a lightweight stack that a small institution can operate without specialist staff.

Chapter 3:

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

CampusEase is organized around the roles of the people who use it and the tasks each role performs. At a high level the system supports four broad activities: (1) **authentication and authorization**, where users register, verify their identity, sign in, and are granted role-specific access; (2) **academic management**, covering enrollment, scheduling, attendance, assignments, marks, and records; (3) **financial and administrative management**, covering fee payment and approval, user and department management, ID cards, and reporting; and (4) **communication and engagement**, covering real-time messaging, discussion forums, events, and clubs. Each activity is exposed through REST API endpoints on the backend and corresponding screens on the Angular frontend, and is restricted to the roles permitted to use it.

3.1.1 Requirement Analysis

Functional Requirements

Table 3.1 lists the functional requirements of CampusEase. The corresponding actor interactions are modelled by the use-case diagram in Figure 3.1.

Table 3.1: Functional requirements of CampusEase.

ID	Requirement	Description
FR-01	Registration & verification	The system shall allow a user to register and verify the account by email, and shall set a password securely.
FR-02	Authentication	The system shall allow a registered user to log in and shall reject invalid credentials.
FR-03	Role-based access	The system shall grant access to features according to the user's role (student, faculty, secretary, administrator, and related staff roles).
FR-04	Subject enrollment	The system shall allow a student to enroll in subjects for a semester using an enrollment key.

ID	Requirement	Description
FR-05	Class scheduling	The system shall allow faculty or staff to create, update, and view class schedules.
FR-06	Attendance	The system shall record attendance through manual entry, one-time password, and face recognition.
FR-07	Assignments	The system shall allow faculty to post assignments and students to submit answers with file uploads.
FR-08	Marks entry	The system shall allow faculty to enter and update student marks.
FR-09	Internal-marks calculation	The system shall compute internal marks automatically from assignment and attendance scores.
FR-10	Fee payment	The system shall allow a student to pay fees online through the Khalti gateway and shall record the payment.
FR-11	Fee approval	The system shall allow an administrator or finance officer to approve or reject fee payments.
FR-12	Messaging	The system shall provide real-time one-to-one messaging between users.
FR-13	Discussion & engagement	The system shall support discussion forums, events, and club membership.
FR-14	Career services	The system shall allow CV submission and job-vacancy posting, and sponsorship requests.
FR-15	ID cards & reports	The system shall generate digital ID cards and provide administrative dashboards and reports.
FR-16	User management	The system shall allow authorized staff to create, update, and delete user accounts.

Non-functional Requirements

Table 3.2 lists the non-functional requirements.

Table 3.2: Non-functional requirements of CampusEase.

ID	Requirement	Description
NFR-01	Performance	The system shall respond to typical requests within a couple of seconds and deliver messages in real time.
NFR-02	Usability	The interface shall be intuitive and operable by a user with basic technical knowledge.
NFR-03	Security	The system shall hash passwords, authenticate requests with JSON Web Tokens, and enforce role-based access.
NFR-04	Reliability	The system shall handle invalid input and unavailable external services gracefully, without crashing.
NFR-05	Maintainability	The codebase shall be modular, separating routes, models, and services on the backend and components and services on the frontend.
NFR-06	Portability	The system shall run on any platform with a Node.js runtime and a MongoDB instance.
NFR-07	Scalability	The system shall support an institution-scale number of users and allow modules to be added without major redesign.

3.1.2 Feasibility Study

Technical Feasibility. The system is technically feasible. It is built with mature, widely used web technologies: Angular for the frontend, Node.js and Express for the backend API, Socket.io for real-time messaging, and MongoDB with Mongoose for storage. Authentication uses JSON Web Tokens and bcrypt password hashing, and payment uses the publicly documented Khalti gateway. All core libraries are open-source and run on commodity hardware without specialized infrastructure.

Operational Feasibility. The system is operationally feasible. It is designed for everyday users, students, faculty, and administrative staff, with role-specific interfaces that guide each user through their tasks. No command-line use or technical configuration is required to operate the system once it is deployed.

Economic Feasibility. The system is economically feasible. The frameworks, database, and libraries are open-source, and the Khalti gateway is a standard local payment service. The platform can be hosted on modest, low-cost infrastructure, so both development and operational costs are minimal.

Schedule Feasibility. The project was planned across an iterative, multi-sprint schedule covering requirement analysis and design, the authentication and role model, the academic modules, the financial and communication modules, and finally integration, testing, and documentation. Building the system module by module allowed a complete, working platform to be delivered within the available time.

3.1.3 Use-Case Modelling

The use-case diagram in Figure 3.1 shows the main actors, students, faculty, and administrators or secretaries, and the use cases each performs, together with the external Khalti payment gateway and email service. Registration includes email verification, paying fees includes a call to the Khalti gateway and can be extended by an administrative approval step, and submitting an assignment relates to the faculty task of posting and grading it.

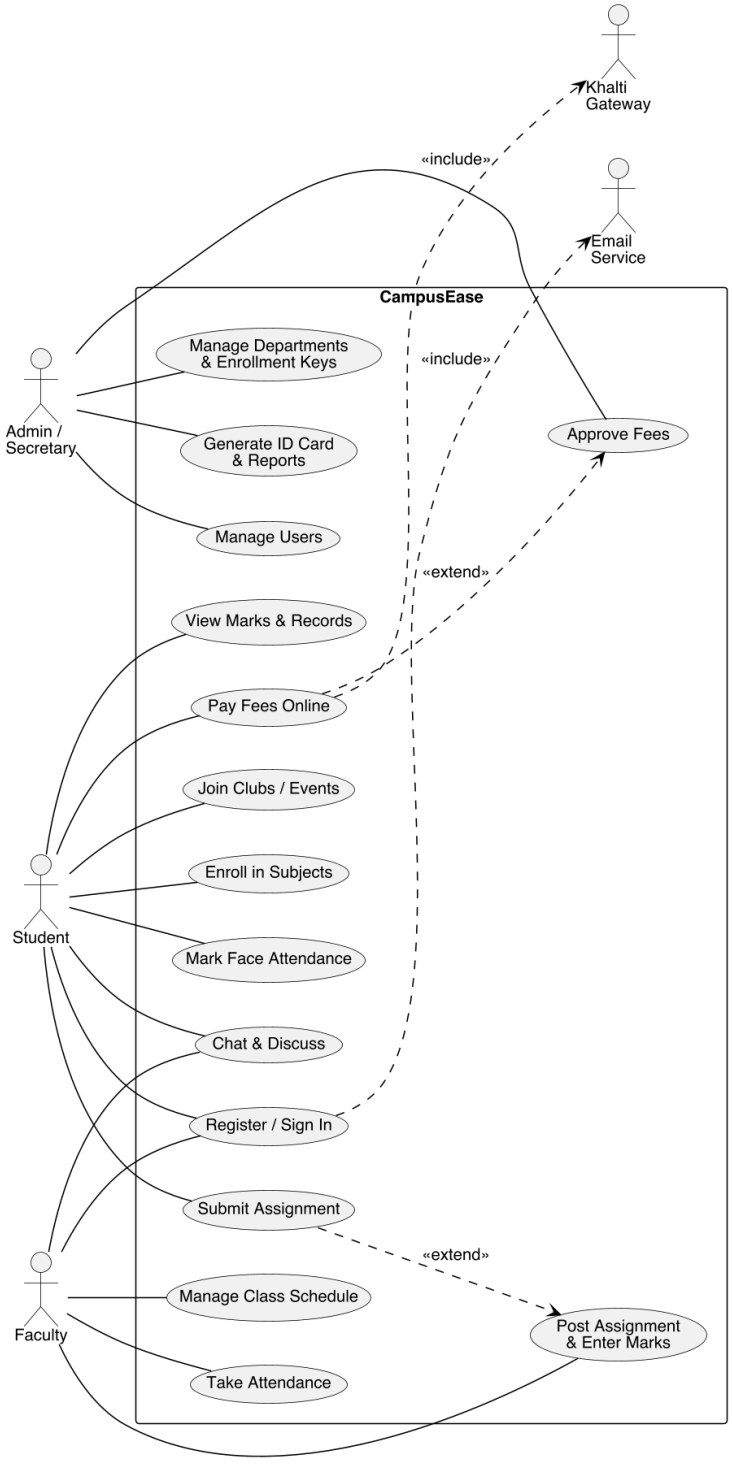

Figure 3.1: Use-case diagram of CampusEase.

3.1.4 Data Modelling: Entity-Relationship Diagram

The data model is stored as MongoDB collections accessed through Mongoose. The core entities are `User`, `EnrollmentSubjects`, `Attendance`, `GiveAssignment`, `AnswerAssignment`,

`MarksEntry`, `Fee`, and `Message`. A student enrolls in the subjects defined by an `EnrollmentSubjects` record; a faculty user posts assignments and enters student marks; a user owns many attendance records, assignment submissions, fee payments, marks, and messages; and each posted assignment is answered by many submissions. Figure 3.2 shows the entity-relationship diagram.

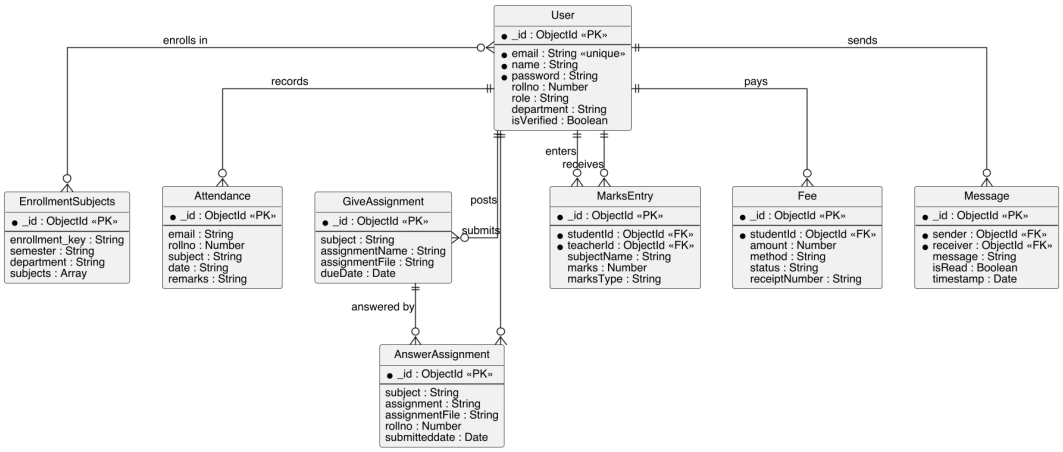

Figure 3.2: Entity-relationship diagram of CampusEase.

3.1.5 Process Modelling: Data Flow Diagrams

The context-level (Level 0) data flow diagram in Figure 3.3 shows CampusEase as a single process exchanging data with its users, the Khalti payment gateway, and the email service.

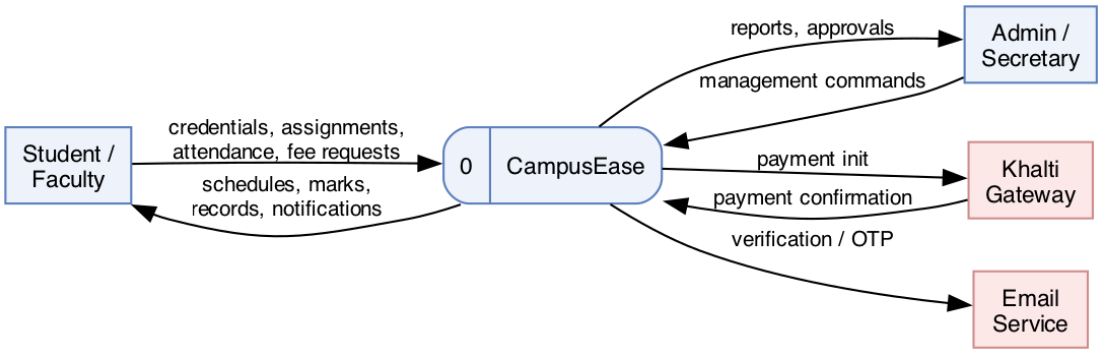

Figure 3.3: Context-level (Level 0) data flow diagram.

The Level 1 data flow diagram in Figure 3.4 decomposes the system into six processes, authenticate and authorize, manage academics, record attendance, manage assignments and marks, process fee payment, and communication and engagement, communicating through six data stores.

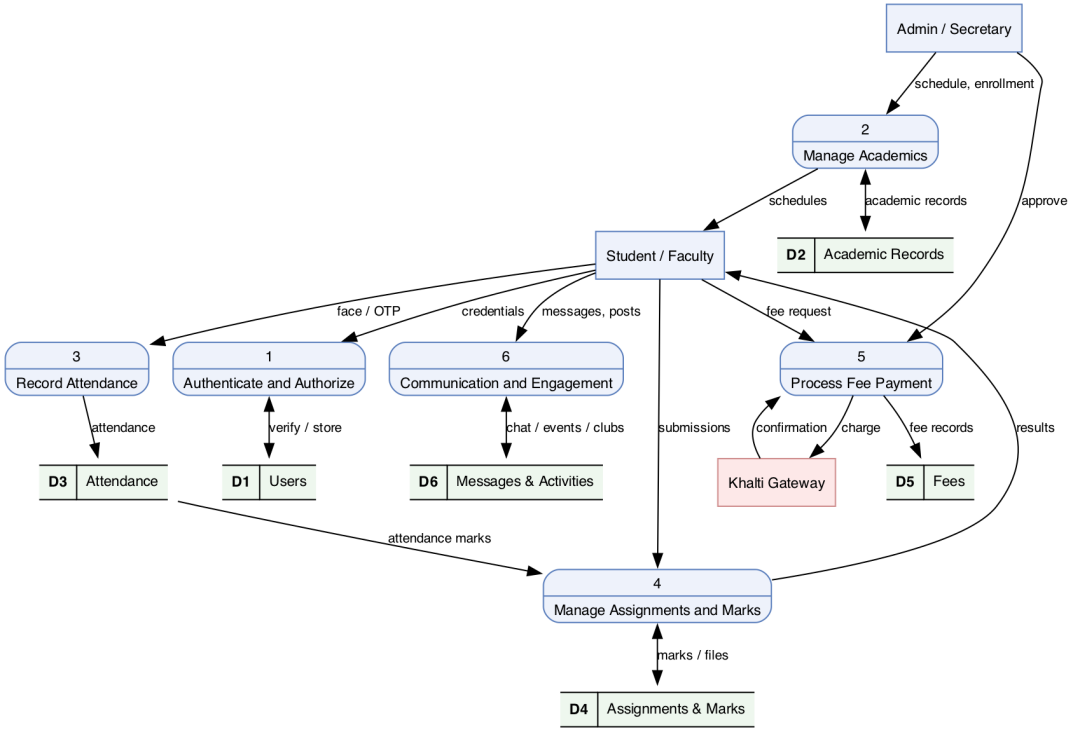

Figure 3.4: Level 1 data flow diagram.

3.2 System Design

3.2.1 Architectural Design

CampusEase follows a layered architecture. The presentation layer is an Angular single-page application running in the browser, with separate portals for students, faculty, and administrators and Highcharts-based dashboards. The application layer is a Node.js and Express REST API, complemented by a Socket.io server for real-time messaging, organized into route groups for authentication, academics, attendance, fees, and communication. The business logic layer contains the JWT authentication and role middleware, the face-recognition service, the internal-marks calculator, the payment handler, the email and one-time-password service, and the socket message hub. The data layer uses the Mongoose object-document mapper over a MongoDB database, and two external services, the Khalti payment gateway and an SMTP mail service, are integrated. Figure 3.5 shows the overall architecture.

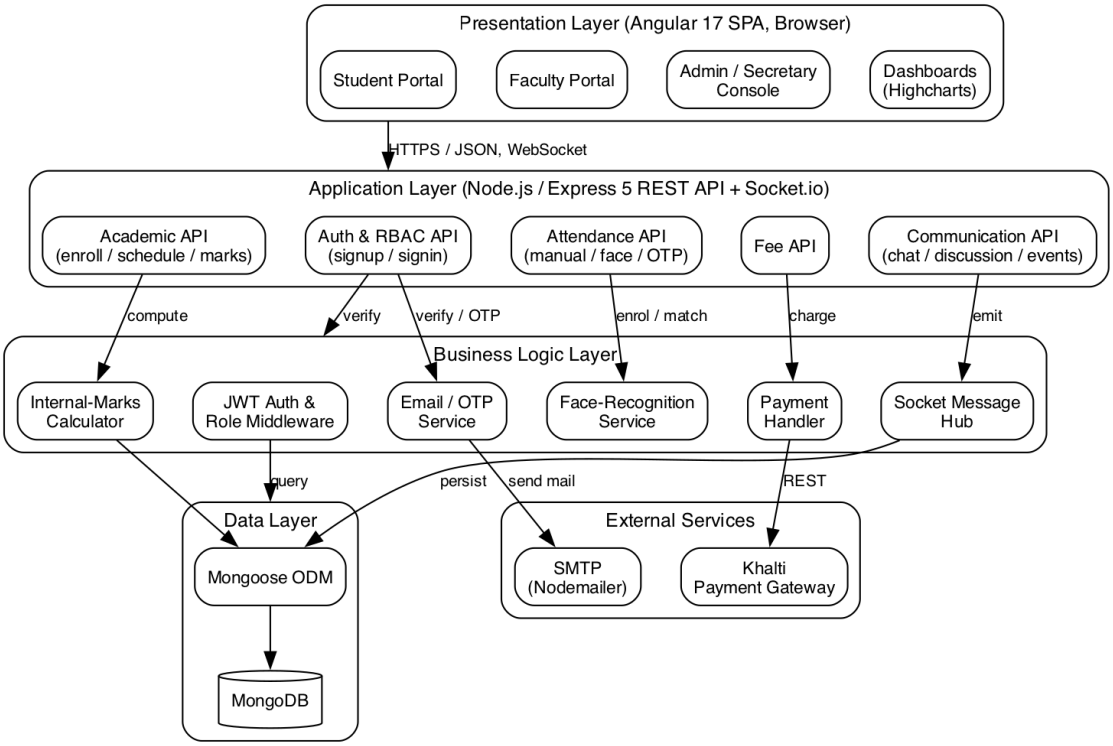

Figure 3.5: Layered system architecture of CampusEase.

3.2.2 Database Schema Design

The database is a set of MongoDB collections defined as Mongoose schemas. Table 3.3 summarizes the key collections and their purpose.

Table 3.3: Database schema summary (key collections).

Collection	Key Fields	Purpose
User (register)	email (unique), name, rollno, password, role, department, isVerified	Stores all user accounts with their role and department for access control.
Enrollment-Subjects	enrollment_key, semester, department, subjects[]	Defines the subjects of a semester and the key students use to enroll.
Attendance / Face	email, rollno, subject, date, remarks; faceEncoding	Stores attendance records, including face-recognition enrollment data.
GiveAssignment / AnswerAssignment	subject, assignmentName, dueDate; rollno, assignmentFile, submitteddate	Stores assignments posted by faculty and submissions made by students.

Collection	Key Fields	Purpose
MarksEntry / InternalMarks	studentId (FK), teacherId (FK), subjectName, marks; assignmentMarks, attendanceMarks	Stores entered marks and the computed internal marks.
Fee	studentId (FK), amount, method, status, receiptNumber	Stores fee payments and their approval status.
Message	sender (FK), receiver (FK), message, isRead, timestamp	Stores real-time chat messages between users.

3.2.3 Interface Design (UI/UX)

The user interface is organized into role-specific areas, designed for clarity and minimal training:

- **Student portal:** enrollment, schedule, attendance, assignment submission, marks and academic records, fee payment, messaging, clubs, and events.
- **Faculty portal:** class scheduling, attendance, assignment posting and grading, marks entry, and model questions.
- **Admin / Secretary console:** user management, fee approval, department and enrollment-key management, ID cards, vacancies, sponsorships, and reports.
- **Dashboards:** summary statistics and charts, built with Highcharts, giving each role an at-a-glance overview.

3.2.4 Activity Modelling

Figure 3.6 shows the activity diagram for face-recognition attendance, one of the system's distinctive features. The student opens the attendance screen; if no face is registered yet, a face is captured and its encoding stored. A live face image is then captured and compared with the stored encoding. If the match confidence is high, attendance is marked present and persisted; otherwise the attempt is rejected and the student is prompted to retry.

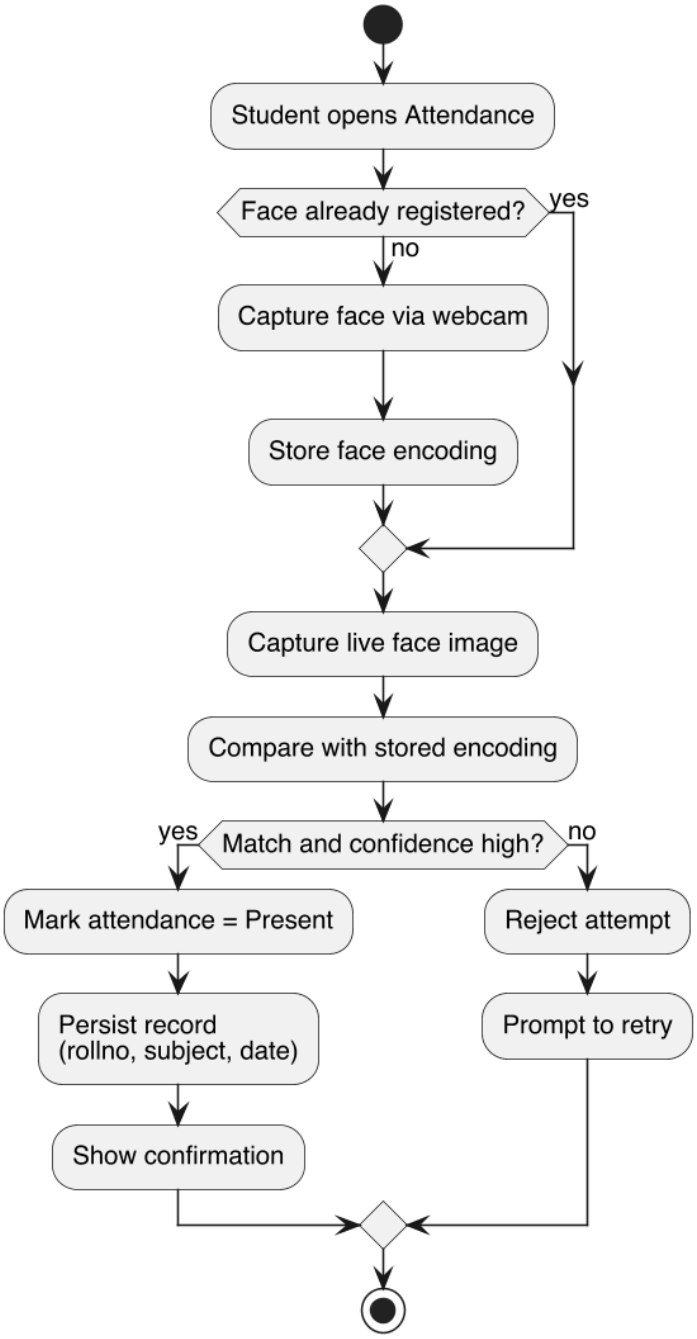

Figure 3.6: Activity diagram for face-recognition attendance.

3.2.5 Sequence Modelling

Figure 3.7 shows the sequence diagram for online fee payment. The student initiates payment from the Angular application, which calls the Khalti gateway; on success the application posts the payment to the Express API, which records a fee with pending status and returns a receipt number. An administrator or finance officer later approves the payment, updating its status to paid.

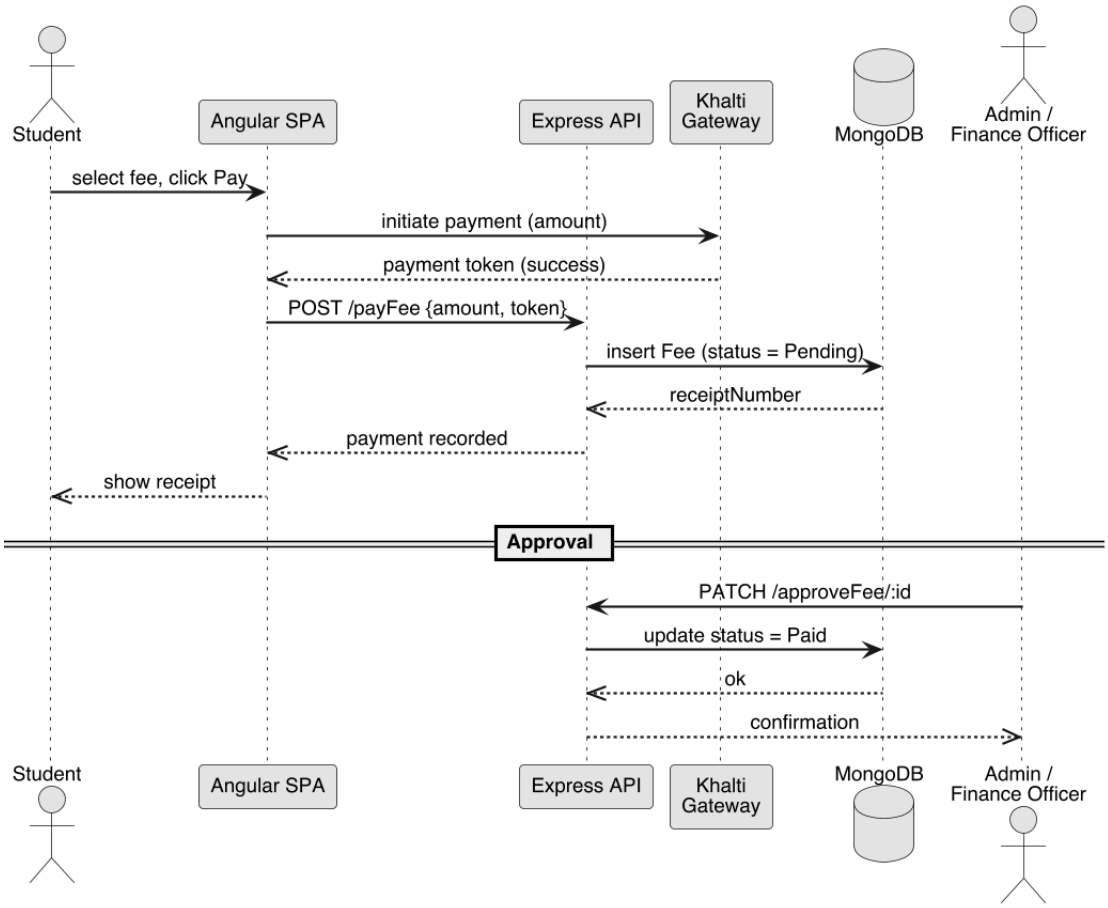

Figure 3.7: Sequence diagram for online fee payment.

Chapter 4:

IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

Development used Visual Studio Code as the editor, Node.js as the runtime, npm for package management, the Angular CLI for the frontend, MongoDB Compass for inspecting the database, and Git and GitHub for version control. Table 4.1 lists the main technologies and the role each plays.

Table 4.1: Technology stack used to build CampusEase.

Technology	**Role in CampusEase**
Angular	Single-page-application framework for the role-specific frontend portals [1].
TypeScript	Typed language used across the Angular frontend for safer, maintainable code.
Bootstrap & ng-bootstrap	Responsive styling and ready-made UI components.
Node.js & Express	JavaScript runtime and web framework providing the REST API [2].
Socket.io	Real-time, bidirectional communication for messaging and presence [5].
MongoDB	NoSQL document database for persistent storage [4].
Mongoose	Object-document mapper that defines schemas and queries MongoDB.
JSON Web Token	Stateless authentication tokens carried in the Authorization header.
bcrypt	One-way password hashing for stored credentials.
Multer	Middleware for handling file uploads (assignments, CVs, faces).
Nodemailer	Sends verification, one-time-password, and reset emails over SMTP.
Khalti	External payment gateway used for online fee payment [6].
Highcharts	Charting library used for dashboards and reports.

4.1.2 Implementation Details of Modules

Authentication and Role Module. This module handles registration, email verification, login, and password management. Passwords are hashed with bcrypt and never stored in plain text. On login the server issues a JSON Web Token, which the client sends with every subsequent request. Middleware verifies the token and checks the user's role and department against the roles permitted for each endpoint, enforcing role-based access control across the system.

Academic Module. The academic module covers subject enrollment, class scheduling, and academic records. A student enrolls in a semester's subjects using an enrollment key; faculty and staff create and maintain class schedules; and academic records and transcripts are stored and retrieved per student.

Attendance Module. Attendance can be recorded in three ways. Manual entry lets faculty mark a class roll directly. One-time-password attendance issues a short-lived code that students enter to confirm presence. Face-recognition attendance captures a student's face through the webcam, stores an encoding on first registration, and on later attempts matches a live capture against the stored encoding before marking the student present.

Assignments and Marks Module. Faculty post assignments with a due date and an optional file; students submit answers with file uploads. Faculty enter marks for each student and subject, and the internal-marks calculator combines assignment marks and attendance marks into a computed internal score, removing a tedious manual calculation.

Fee Payment Module. Students pay fees online through the Khalti gateway. The frontend initiates the payment, and on success the backend records a fee with a receipt number and a pending status. An administrator or finance officer then approves or rejects the payment, and the status is updated accordingly. Payments by other methods are recorded and approved manually.

Communication and Engagement Module. Real-time one-to-one messaging is implemented with Socket.io: each connected user registers their socket, online presence is tracked, and messages are delivered instantly to the recipient and persisted to MongoDB. Discussion forums, events, and club membership round out the engagement features.

Administration and Career Module. Authorized staff manage users, departments, and enrollment keys, approve fees and ID-card requests, and post job vacancies. Students submit CVs and sponsorship requests and generate digital ID cards. Dashboards built with Highcharts summarize activity for each role.

4.2 Testing

Because CampusEase is built module by module, testing was carried out module by module and then end to end. Each feature was tested with representative inputs as it was completed, and the integrated system was exercised through the user interface for each role.

4.2.1 Test Cases for Unit Testing

Table 4.2: Unit test cases.

ID	Test Case	Expected Result	Status
UT-01	Register with a new email	Account created, verification email sent	Pass
UT-02	Register with an existing email	Registration rejected with a clear message	Pass
UT-03	Hash and verify password	Correct password verifies, incorrect one does not	Pass
UT-04	Issue and verify JWT	Valid token is accepted, tampered token is rejected	Pass
UT-05	Role check on a protected route	A role without permission is denied access	Pass
UT-06	Enroll using a valid enrollment key	Student enrolled in the semester's subjects	Pass
UT-07	Mark manual attendance	Attendance record stored with correct fields	Pass
UT-08	Face-recognition match	A matching face marks attendance present	Pass
UT-09	Submit an assignment file	Submission stored with student and subject	Pass
UT-10	Calculate internal marks	Assignment and attendance marks combine correctly	Pass
UT-11	Record a fee payment	Fee stored with pending status and receipt number	Pass
UT-12	Approve a fee payment	Fee status updates to paid	Pass
UT-13	Send a real-time message	Message delivered to recipient and persisted	Pass
UT-14	Generate a digital ID card	ID card created for an approved request	Pass

4.2.2 Test Cases for System Testing

Table 4.3: System (end-to-end) test cases.

ID	Test Case	Expected Result	Status
ST-01	Register, verify, and log in as a student	Student reaches the student portal	Pass
ST-02	Log in with wrong credentials	Login rejected with a clear message	Pass
ST-03	Enroll in subjects and view schedule	Subjects and schedule shown correctly	Pass
ST-04	Take attendance by face and by OTP	Both methods record attendance	Pass
ST-05	Post and submit an assignment	Faculty posts, student submits, file stored	Pass
ST-06	Enter marks and view internal marks	Marks entered, internal marks computed	Pass
ST-07	Pay a fee online and approve it	Payment recorded, then approved by staff	Pass
ST-08	Send messages between two users	Messages delivered in real time	Pass
ST-09	Manage users as an administrator	Create, update, and delete users work	Pass
ST-10	Role-based access enforcement	Each role can access only permitted areas	Pass
ST-11	Post a vacancy and submit a CV	Vacancy listed, CV stored and listed	Pass
ST-12	View dashboard charts	Charts render with correct summary data	Pass

4.2.3 Results and Discussion

All unit and system test cases passed. Registration, email verification, and login behaved correctly, and the JSON-Web-Token and role checks reliably allowed permitted actions while rejecting unauthorized ones. The academic modules worked end to end: students enrolled with a key, attendance was recorded by all three methods, assignments were posted and submitted, and internal marks were computed correctly from assignment and attendance scores. Online fee payment recorded a pending fee that staff could approve, and real-time messaging delivered messages instantly while persisting them for later retrieval. The administrative features, user management,

vacancies, CV submission, ID cards, and dashboards, all functioned as expected. The main observation is that building the role model and shared data model first paid off: because every later module relied on them, getting authentication and access control right early made the remaining modules straightforward to add and test. The overall control and data flow of the system is summarized in the flowchart in Figure 4.1.

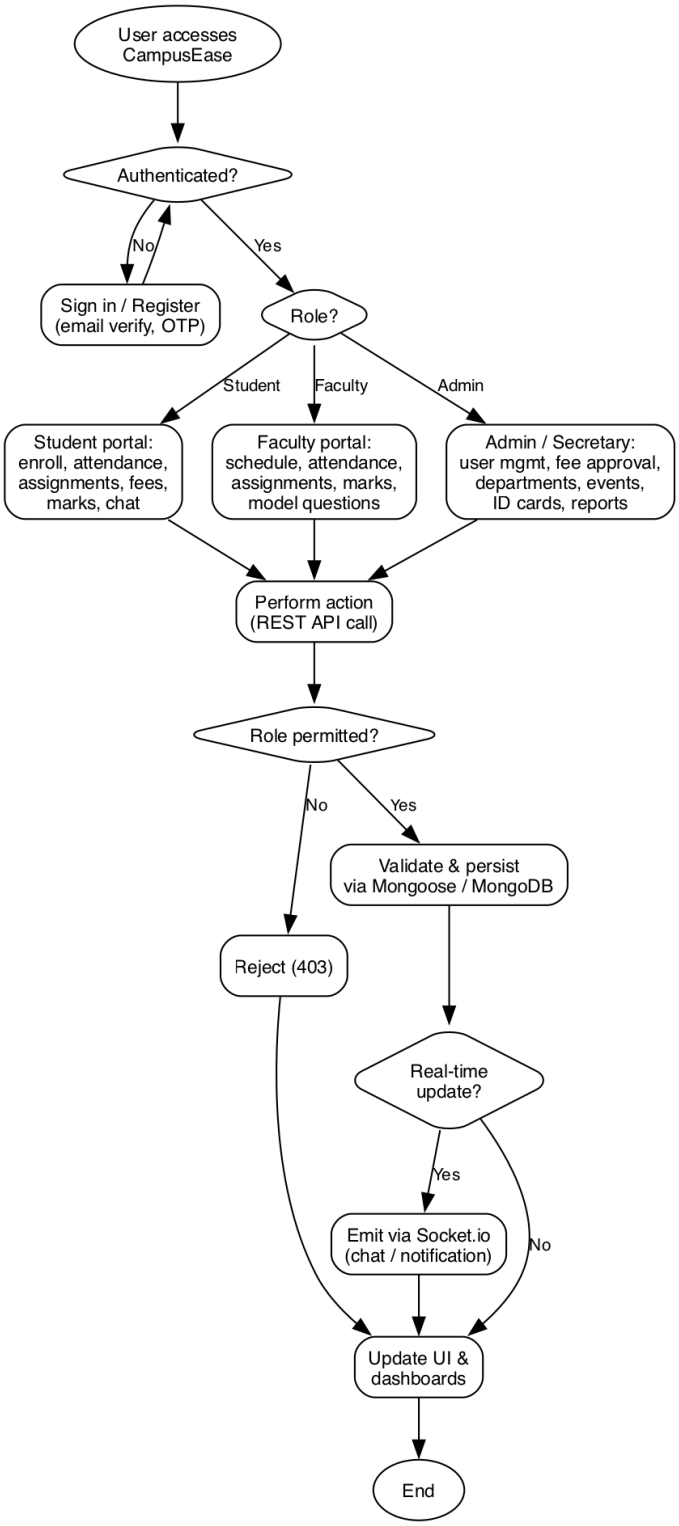

Figure 4.1: System flowchart of CampusEase.

Chapter 5:

CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

This project set out to replace the manual, fragmented processes of a college with a single, integrated, role-aware web platform, and it achieves that goal. CampusEase is a complete, working application that unifies authentication and role-based access, subject enrollment and scheduling, attendance by manual, one-time-password, and face-recognition methods, assignment distribution and submission, marks entry and automatic internal-marks calculation, online fee payment with administrative approval, and real-time communication through messaging, discussions, events, and clubs, along with career services, sponsorship, and digital ID cards. Built with an Angular frontend, a Node.js and Express API with Socket.io, and MongoDB through Mongoose, secured with JSON Web Tokens and bcrypt, the system reduces administrative workload, minimizes errors, and improves transparency and communication. Every objective set out in Chapter 1 was met, and testing confirmed that each module and the system as a whole behave correctly.

5.2 Lesson Learnt / Outcome

The project reinforced several lessons. First, designing the role-based access model and the shared user data model before the feature modules paid off, because almost every later feature depended on them. Second, an iterative, module-by-module approach made a large, multi-feature system manageable and allowed continuous integration and testing. Third, integrating external services such as the Khalti payment gateway and an email provider showed the importance of handling failures gracefully so that the rest of the system keeps working. The outcome is a deployable, multi-role college automation platform together with practical experience in full-stack web development, real-time systems, secure authentication, and third-party integration.

5.3 Future Recommendations

Several enhancements would extend the system beyond its current scope:

- **Mobile application** for students and faculty, reusing the existing API, for fully on-the-go access.
- **Push and email notifications** for high-priority events such as new marks, fee due dates, and announcements.
- **Advanced analytics** on attendance, performance, and fee collection to support data-driven decisions.
- **Production-grade security** with refresh-token rotation, centralized session management, and rate limiting.
- **Scalable deployment** with a replicated MongoDB cluster and horizontal scaling of the API for multi-campus use.
- **More payment options** and automatic reconciliation alongside the existing Khalti integration.

APPENDICES

Appendix A: Application Screenshots

The following screenshots show the running CampusEase application.

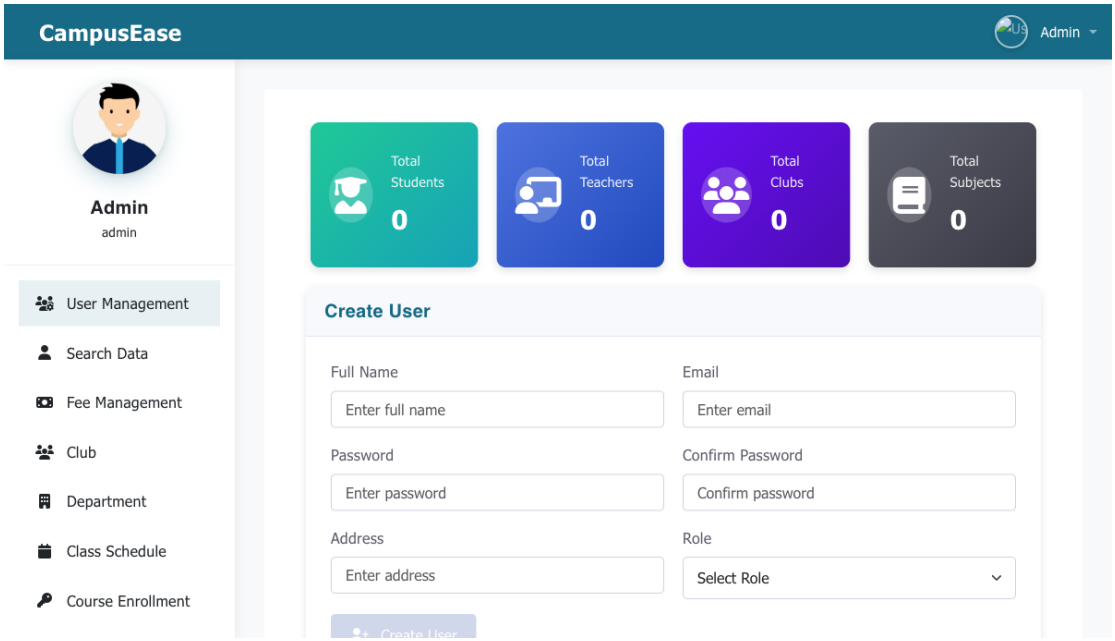

Figure A.1: Administrator console: summary cards and the create-user panel (empty institution).

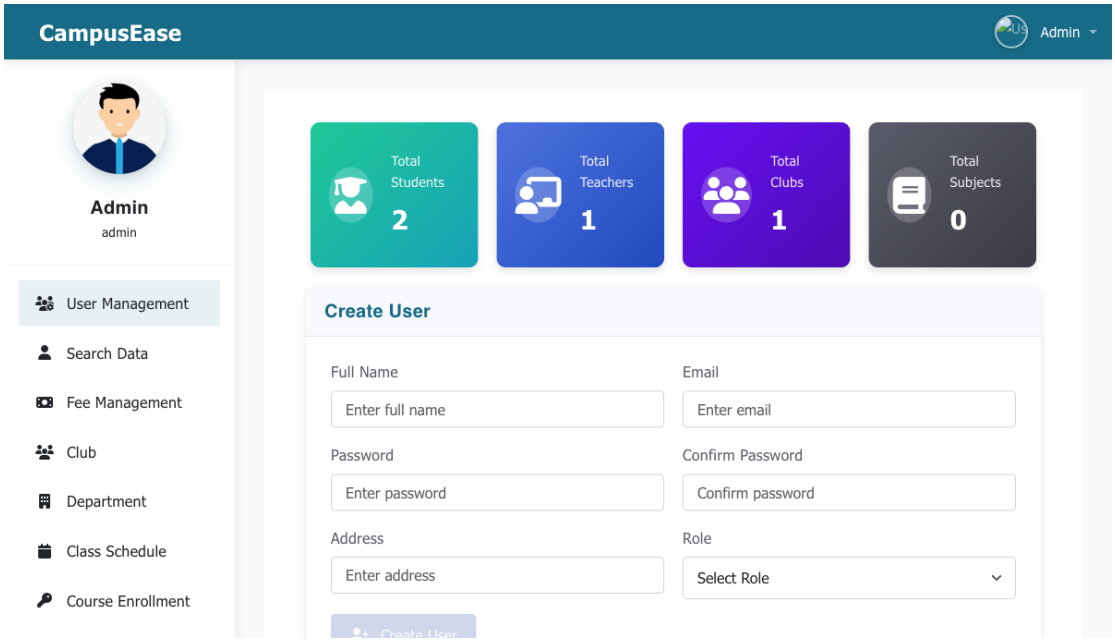

Figure A.2: User-management screen with populated student, teacher, and club counts.

Appendix B: Representative Source Code

The role-based access control middleware that protects the API is reproduced below.

```
const jwt = require("jsonwebtoken");

// Verify the JSON Web Token sent in the Authorization header
function verifyToken(req, res, next) {
  const header = req.headers["authorization"] || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : header;
  if (!token) return res.status(401).json({ message: "No token provided" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

// Allow the request only if the user's role is in the allowed list
function verifyRole(allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: "Access denied" });
```

```
    }
    next();
  };
}

module.exports = { verifyToken, verifyRole };
```

Project Log Book Entry Sheet

Meeting No.: 01 Date: 22/11/2025

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Introduction to final year project requirements and evaluation criteria
2. Project phases, deadlines, and submission milestones
3. Overview of the report chapters: Introduction, Literature Review, System Analysis and Design, Implementation and Testing, and Conclusion
4. IEEE format, abstract writing, and reference guidelines

Achievements:

1. Understood the complete project roadmap
2. Received the project guideline document

Problems (if any):

1. None

Task for next meeting:

1. Brainstorm and shortlist project ideas

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 02 Date: 29/11/2025

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Presented the shortlisted project ideas
2. Finalized "CampusEase: a web-based college automation system"
3. Discussed the problem statement, objectives, and significance

Achievements:

1. Project title approved by the supervisor
2. Problem statement and objectives drafted

Problems (if any):

1. None

Task for next meeting:

1. Write the Introduction chapter
2. Carry out the feasibility study

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 03 Date: 13/12/2025

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. How to write the literature review
2. Citation and referencing in IEEE format
3. Reviewing existing systems (Google Classroom, Moodle, campus ERPs)

Achievements:

1. Learned the IEEE citation style
2. Collected reference papers and compared existing platforms

Problems (if any):

1. None

Task for next meeting:

1. Complete the literature review draft
2. Prepare the reference list

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 04 Date: 27/12/2025

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Reviewed the Introduction and Literature Review chapters
2. Finalized the functional and non-functional requirements
3. Feasibility study (technical, operational, economic, and schedule)

Achievements:

1. Chapters 1 and 2 completed
2. Requirements and feasibility confirmed

Problems (if any):

1. None

Task for next meeting:

1. Start the system design and UML modelling
2. Finalize the technology stack

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 05 Date: 10/01/2026

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. System analysis and design chapter writing
2. Diagrams: use case, ER, data flow, sequence, activity, and architecture
3. Project proposal format and mid-defense preparation

Achievements:

1. Drafted the use-case and ER diagrams
2. Understood the proposal structure

Problems (if any):

1. None

Task for next meeting:

1. Complete all the diagrams
2. Submit the project proposal

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 06 Date: 24/01/2026

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Reviewed the use-case, ER, and data flow diagrams
2. Technology stack confirmation (Angular, Node.js and Express, MongoDB, Socket.io)

Achievements:

1. All diagrams approved by the supervisor
2. Development environment set up (VS Code, Node.js, Git and GitHub)

Problems (if any):

1. Confusion in showing ER relationships (one-to-many vs many-to-many); the supervisor helped to correct them

Task for next meeting:

1. Start frontend development (registration and login)
2. Set up the Angular and Express project structure

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 07 Date: 07/02/2026

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Registration, login, and JWT authentication progress
2. Role-based dashboards (student, faculty, secretary, and administrator)
3. Attendance methods (manual, OTP, and face recognition)

Achievements:

1. Authentication and role-based access working
2. MongoDB connected and the Khalti fee payment integrated
3. Separate dashboards completed for each role

Problems (if any):

1. Face-recognition matching needed tuning; resolved by adjusting the confidence threshold

Task for next meeting:

1. Implement the real-time messaging module
2. Prepare for the mid defense

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 08 Date: 07/03/2026

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Mid-defense presentation to the supervisor and examiners
2. Live demo (registration, login, enrollment, attendance, fee payment, and messaging)
3. Questions and feedback

Achievements:

1. Mid defense cleared successfully
2. Supervisor appreciated the working prototype
3. Received constructive feedback

Problems (if any):

1. The user interface was pointed out as too plain and needing polish
2. The examiners noted that the test cases were not sufficient
3. A question on multi-campus scalability was raised; explained as future scope

Task for next meeting:

1. Improve the user-interface design
2. Add more unit and system test cases
3. Complete the remaining report chapters

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 09 Date: 28/03/2026

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Final report review (all chapters)
2. Abstract, conclusion, and future recommendations
3. Final-defense presentation and viva preparation

Achievements:

1. Final report draft completed in LaTeX
2. All diagrams inserted correctly
3. Abstract and conclusion finalized and slides prepared

Problems (if any):

1. Figure placement in LaTeX was difficult; solved using the [H] option with the float package
2. Formatting the test-case tables was hard; solved using longtable

Task for next meeting:

1. Submit the final report
2. Prepare answers for the viva

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

Project Log Book Entry Sheet

Meeting No.: 10 Date: 09/05/2026

Start Time: 10:00 AM Finish Time: 11:00 AM

Discussion Topics:

1. Final-defense schedule and date confirmation
2. Rules and regulations, presentation duration, and dress code
3. Documents to bring, backup slides, and live-demo guidelines

Achievements:

1. Understood all the rules and regulations for the final defense
2. Prepared a checklist for defense day

Problems (if any):

1. None

Task for next meeting:

1. Appear for the final defense
2. Submit the hard copy of the report

Student Name: Bhupendra Malla Mentor Signature

Signature: ....

REFERENCES

[1] Google, "Angular Documentation," 2025. [Online]. Available: https://angular.dev/

[2] OpenJS Foundation, "Express Web Framework Documentation," 2025. [Online]. Available: https://expressjs.com/

[3] OpenJS Foundation, "Node.js Documentation," 2025. [Online]. Available: https://nodejs.org/en/docs

[4] MongoDB Inc., "MongoDB Manual," 2025. [Online]. Available: https://www.mongodb.com/docs/

[5] Socket.io, "Socket.io Documentation," 2025. [Online]. Available: https://socket.io/docs/v4/

[6] Khalti, "Khalti Payment Gateway Documentation," 2025. [Online]. Available: https://docs.khalti.com/

[7] Moodle Pty Ltd., "Moodle Documentation," 2025. [Online]. Available: https://docs.moodle.org/

[8] Google, "Google Classroom Help," 2025. [Online]. Available: https://support.google.com/edu/classroom/

[9] Internet Engineering Task Force, "RFC 7519: JSON Web Token (JWT)," 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[10] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org/